

National University of Computer and Emerging Sciences



Lab Manual 02

Computer Organization
and Assembly Language

Course Instructor	Mr. Salman Mubarik
Lab Instructor	Muhammad Hasan
Lab Instructor Email ID	m.hasan@lhr.nu.edu.pk
Semester	Fall 2026

FAST - School of Computing
FAST-NU, Lahore Campus

Contents

Objectives:.....	3
Activities:	3
Activity 1 (Some Important Concepts):.....	3
• Addressing Modes	3
• Data Types	4
• Arrays	4
• Loops	4
• Jumps	5
• How to view Memory in AFD	5
Activity 2 (Answering Theoretical Questions):	6
Activity 3 (Assembly Programming Tasks):	6

Objectives:

- To understand and apply basic addressing modes in Assembly.
- To understand data types and memory declaration using DB, DW, DD, DQ, and DT.
- To work with arrays, loops, and jump instructions.
- To view and manipulate memory using AFD memory windows.
- To analyze and understand processor flags.
- To practice basic Assembly programs using registers and memory.

Activities:

Activity 1 (Some Important Concepts):

- *Addressing Modes*

An addressing mode specifies how an Assembly Language instruction identifies or obtains its operand (data). The operand may be located in a register, within the instruction itself, or in memory. Different addressing modes provide different ways of accessing data efficiently.

Addressing Mode	Operand Location / How Address is Obtained	Example	Meaning
Register Addressing	Operand is stored in a CPU register.	MOV AX, BX	Copy the value in BX into AX.
Immediate Addressing	The actual constant value is included directly in the instruction.	MOV AX, 5	Put the value 5 into AX.
Direct Addressing	The memory address of the operand is explicitly specified in the instruction.	MOV AX, [1000h]	Copy the value stored at memory address 1000h into AX.
Register Indirect Addressing	A register contains the memory address of the operand.	MOV AX, [BX]	Use the value in BX as a memory address and load that memory value into AX.
Indexed Addressing	A memory address is obtained using an index register, optionally with a displacement.	MOV AX, [SI+10]	Access the memory location at address SI + 10 and load its value into AX.
Based Addressing	A base register is used with an optional displacement to obtain the memory address.	MOV AX, [BX+10]	Access memory at address BX + 10.
Based-Indexed Addressing	A base register and an index register are combined to calculate the memory address.	MOV AX, [BX+SI]	Access memory at address BX + SI.
Based-Indexed with Displacement	Base register + index register + displacement are used to calculate the memory address.	MOV AX, [BX+SI+10]	Access memory at address BX + SI + 10.

- **Data Types**

In Assembly Language, data types define the amount of memory reserved for storing data. Variables are declared in memory using data-definition directives such as *DB*, *DW*, *DD*, *DQ*, and *DT*. Each directive reserves a specific number of bytes and therefore determines the range of values that can be stored.

Directive	Full Form	Memory Allocated	Unsigned Value Range
DB	Define Byte	1 byte (8 bits)	$0 - (2^8 - 1) = 0 - 255$
DW	Define Word	2 bytes (16 bits)	$0 - (2^{16} - 1) = 0 - 65,535$
DD	Define Doubleword	4 bytes (32 bits)	$0 - (2^{32} - 1) = 0 - 4,294,967,295$
DQ	Define Quadword	8 bytes (64 bits)	$0 - (2^{64} - 1)$
DT	Define Ten Bytes	10 bytes (80 bits)	$0 - (2^{80} - 1)$

Note: The ranges shown above are for **unsigned values**. Signed values have a different range because one bit is used to represent the sign.

Examples:

```
num1 DB 25    ; 1 byte
num2 DW 1000   ; 2 bytes
num3 DD 50000  ; 4 bytes
num4 DQ 100000 ; 8 bytes
num5 DT 12345  ; 10 bytes
```

- **Arrays**

An array is a collection of elements of the same data type stored in consecutive memory locations. Arrays allow multiple values to be stored and accessed using a common base address and an index or offset. In Assembly Language, arrays are commonly accessed using register indirect or indexed addressing.

Example:

```
numbers DB 10, 20, 30, 40, 50
```

- **Loops**

A loop is used to repeatedly execute a block of instructions until a specified condition is met or a counter reaches its limit. Loops reduce code repetition and are commonly used for array traversal, counting, and repetitive calculations. In 8086 Assembly, the LOOP instruction is commonly used with the CX register as a counter.

Example:

```
[org 0x0100]

MOV CX, 5    ; CX = loop counter
MOV AX, 0    ; AX = running sum
MOV BX, 1    ; BX = current number

L1:
  ADD AX, BX ; AX = AX + BX
  INC BX    ; Move to the next number
  LOOP L1   ; CX = CX - 1; repeat if CX != 0

MOV AX, 0x4C00 ; Terminate program
INT 0x21
```

- *Jumps*

A jump changes the normal sequential flow of program execution by transferring control to another instruction identified by a label. Jumps can be unconditional, such as JMP, or conditional, such as JE, JNE, JG, and JL, which make decisions based on processor flags.

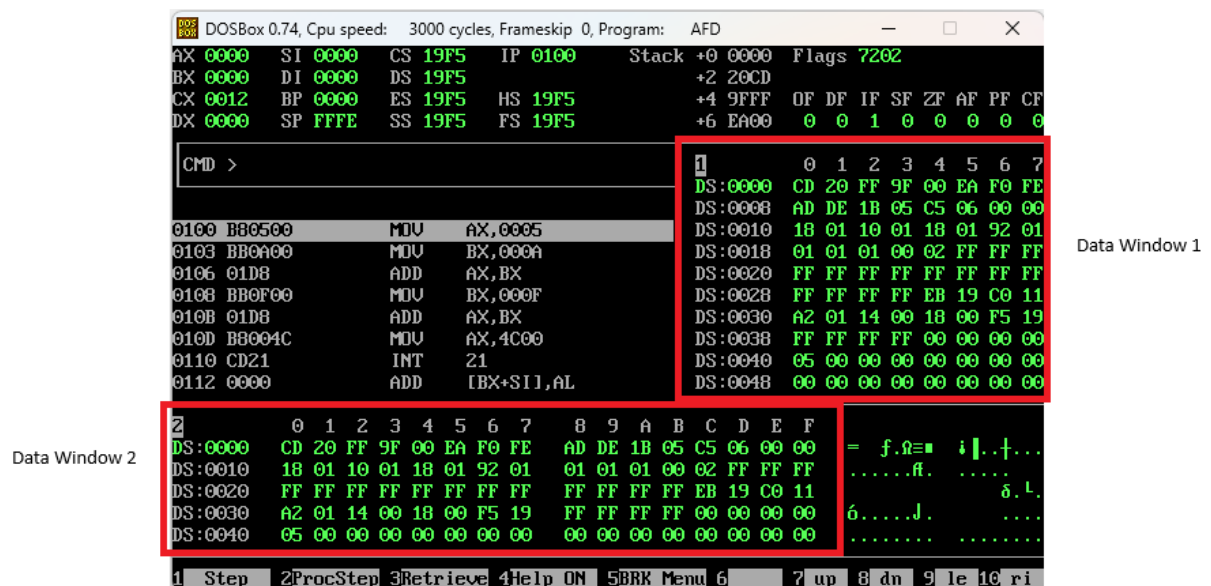
Examples:

JMP NEXT ; Unconditional jump

CMP AX, BX

JE EQUAL ; Jump if AX = BX

- *How to view Memory in AFD*



Memory windows in AFD allow us to view the contents of memory at a specific segment and offset. Each memory location stores a byte, represented in hexadecimal. AFD provides two memory windows, m1 and m2, so that two different memory locations can be viewed simultaneously.

Memory Address Format

A memory location in AFD is specified using:

Segment:Offset

For example:

DS:0040 means:

DS → Data Segment

0040 → Offset within the Data Segment

Therefore, AFD displays the byte stored at that particular memory location.

To display memory in Data Window 1, use:

m1 DS:0040

Or to display memory in Data Window 2, use:

m2 DS:0040

This tells AFD to display the memory contents starting from offset 0040h in the Data Segment (DS) in Memory Window.

Note: m1 and m2 are simply two separate memory-display windows. They do not represent different types of memory.

Activity 2 (Answering Theoretical Questions):

Answer the following questions in a text file.

- i Which addressing mode is typically the fastest and involves only CPU registers?
- ii What addressing mode uses a constant or literal value directly within the instruction?
- iii When would you use direct addressing mode in x86 assembly?
- iv How does indirect addressing mode provide flexibility in accessing data?
- v Which registers are commonly used in indirect addressing mode?
- vi What addressing mode is ideal for initializing registers with specific values?
- vii In indexed addressing mode, what two components are combined to form the effective address?
- viii Why is indirect addressing mode useful for accessing dynamic data locations?
- ix Which addressing mode is particularly suitable for navigating arrays and data structures?
- x What is the main benefit of using register addressing mode?

Activity 3 (Assembly Programming Tasks):

1. Write instructions to do the following. Visualize the memory contents using memory windows to see if instruction is executed correctly. (Use m2 DS:offset to visualize the memory contents at the specified offset)
 - i Copy contents of memory location with offset 0025 into AX.
 - ii Copy AX into memory location with offset 0FFF.
 - iii Move contents of memory location with offset 0010 to memory location with offset 002F.
2. Using the dw directive, declare a 16-bit variable initialized to 0x1234, then write an assembly snippet that swaps its high-order and low-order bytes in the original variable so that the original left byte becomes the right byte and the original right byte becomes the left byte. (use indirect addressing).
3. Use the db and dw directives to set up a 6-byte source array and a 6-word destination array. Using registers and an explicit offset, write a loop that reverses the 6-byte source array into the 6-word destination array.
4. Give the value of the zero flag, the carry flag, the sign flag, and the overflow flag after each of the following instructions:

Instructions	ZF	CF	SF	OF
mov ax, 0x1254				
mov bx, 0x0FFF				
add ax, 0xEDAB				
add ax, bx				
add bx, 0xF001				

5. Write a program that computes arithmetic sum of first N numbers in AX where N is a memory variable. If N = 6, your program should calculate $6+5+4+3+2+1 = 21$ in AX. Test your program on different numbers.
6. Find frequency of an integer from an array of 10 integers. For example, frequency of 2 in array 2, 4, 2, 8, 2 is 3
7. Write a program that computes square of 'num' and saves it in 'square'. Where num and square are two memory variables of word length. For any number n, your program should

add n, n times to produce its square. For example ($5^2 = 5+5+5+5+5$). Test your program on num = 5 and num = 9.

8. Write a program to generate first 10 terms of the Fibonacci Series. The generated terms are to be placed at memory location named "Fib" using indirect addressing.
9. Write a program that reads array1 and saves it in array2 in reverse order. Sample run is given below.

Array1: 1, 2, 3, 4, 5, 6

Array2: 0, 0, 0, 0, 0, 0

After Program Execution:

Array1: 1, 2, 3, 4, 5, 6

Array2: 6, 5, 4, 3, 2, 1

Hint: You may need SI and DI (Source Index and Destination Index) registers to save two different indices.